

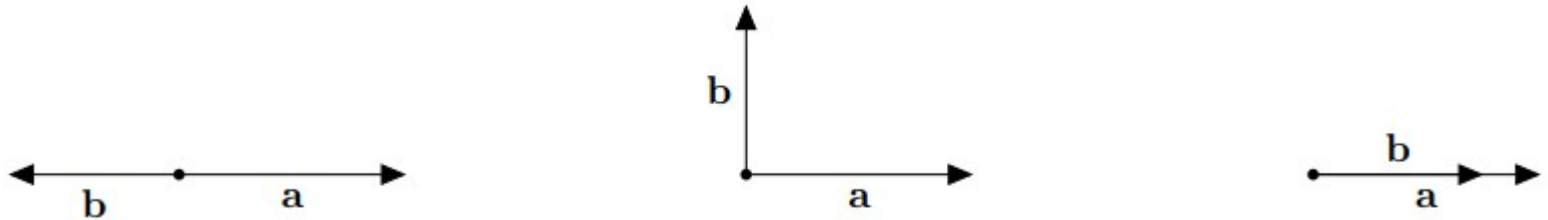


Visible Surface Detection

Instructor: Dinesh Maharjan

2022

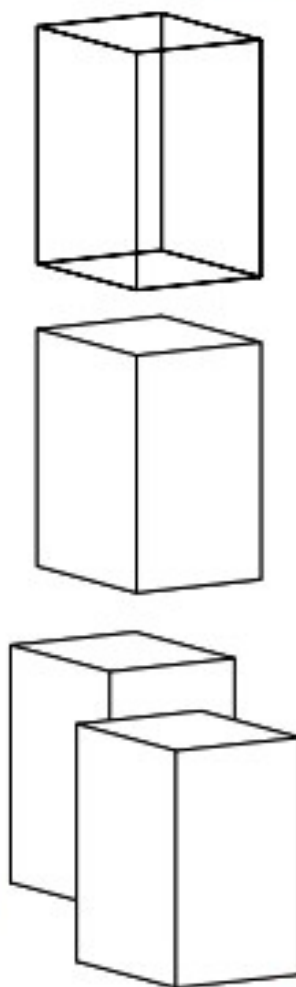
Dot product intuition



- First figure, dot product < 0
- Second figure, dot product $= 0$
- Third figure, dot product > 0

Visible-Surface Detection

- identifying visible parts of a scene (also *hidden-surface elimination*)
- type of algorithm depends on:
 - ◆ complexity of scene
 - ◆ type of objects
 - ◆ available equipment
 - ◆ static or animated displays
- *object-space* methods
 - ◆ objects compared to each other
- *image space* methods
 - ◆ point by point at each pixel location
- often sorting and coherence used





Visible Surface Detection

- Detects edges or surfaces that are visible to observer.
- Then render lines or surfaces to produce realistic scene.
- Broadly classified into
 - Image space methods
 - Object space methods
 -



Image Space Method

- Visibility is decided point by point
- Done after projection
- Less expensive
- Change in resolution requires recomputation
- For each pixel in image
 - Determine the pixel closest to viewer
 - Draw the pixel
- For e.g. Depth Buffer Method



Object Space Method

- Objects and parts of objects are compared
- Done before projection
- Device independent
- Computationally expensive
- Suitable for simple scenes with less objects
- For each object in the world
 - Determine the blocked part of object
 - Draw those parts in appropriate color
- For e.g. Back-face Detection

Cross Product

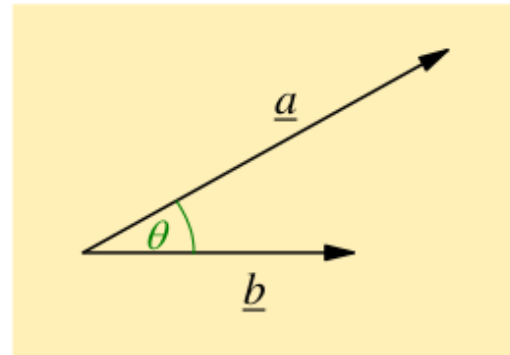
The purpose of this tutorial is to practice working out the vector product of two vectors.

It is called the 'vector product' because the result is a 'vector', i.e. a quantity with both (i) **magnitude**, and (ii) **direction**.

(i) The **MAGNITUDE** of the **vector product** of $\underline{a}$ and $\underline{b}$ is

$$|\underline{a} \times \underline{b}| = |\underline{a}||\underline{b}| \sin \theta$$

where θ is the angle
between $\underline{a}$ and $\underline{b}$.



Cross Product

$$\begin{aligned}\underline{a} \times \underline{b} &= \begin{vmatrix} \underline{i} & \underline{j} & \underline{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix} \\ &= \underline{i} \begin{vmatrix} a_y & a_z \\ b_y & b_z \end{vmatrix} - \underline{j} \begin{vmatrix} a_x & a_z \\ b_x & b_z \end{vmatrix} + \underline{k} \begin{vmatrix} a_x & a_y \\ b_x & b_y \end{vmatrix}\end{aligned}$$

Cross Product

Here, we have $\underline{a} = 2\underline{i} + \underline{j} - \underline{k}$ and $\underline{b} = 3\underline{i} - 6\underline{j} + 2\underline{k}$

$$\begin{aligned}\underline{a} \times \underline{b} &= \begin{vmatrix} \underline{i} & \underline{j} & \underline{k} \\ 2 & 1 & -1 \\ 3 & -6 & 2 \end{vmatrix} \\ &= \underline{i} \begin{vmatrix} 1 & -1 \\ -6 & 2 \end{vmatrix} - \underline{j} \begin{vmatrix} 2 & -1 \\ 3 & 2 \end{vmatrix} + \underline{k} \begin{vmatrix} 2 & 1 \\ 3 & -6 \end{vmatrix} \\ &= \underline{i}[(1)(2) - (-1)(-6)] - \underline{j}[(2)(2) - (-1)(3)] + \underline{k}[(2)(-6) - (1)(3)] \\ \underline{a} \times \underline{b} &= \underline{i}[2 - 6] - \underline{j}[4 + 3] + \underline{k}[-12 - 3] \\ &= -4\underline{i} - 7\underline{j} - 15\underline{k}.\end{aligned}$$



Cross Product

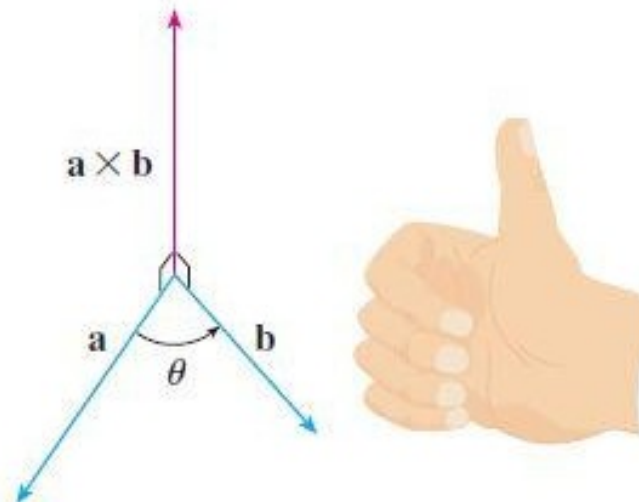
- Dot product of $\mathbf{a} \times \mathbf{b}$ and $\mathbf{a}$ is
 - $-8 - 7 + 15 = 0$
- Dot product of $\mathbf{a} \times \mathbf{b}$ and $\mathbf{b}$ is
 - $-12 + 42 - 30 = 0$
- This shows that $\mathbf{a} \times \mathbf{b}$ is perpendicular to $\mathbf{a}$ and $\mathbf{b}$.
- This helps us to calculate normal vector to a surface.

Orthogonal Vectors

When you take the cross product of two vectors **a** and **b**,

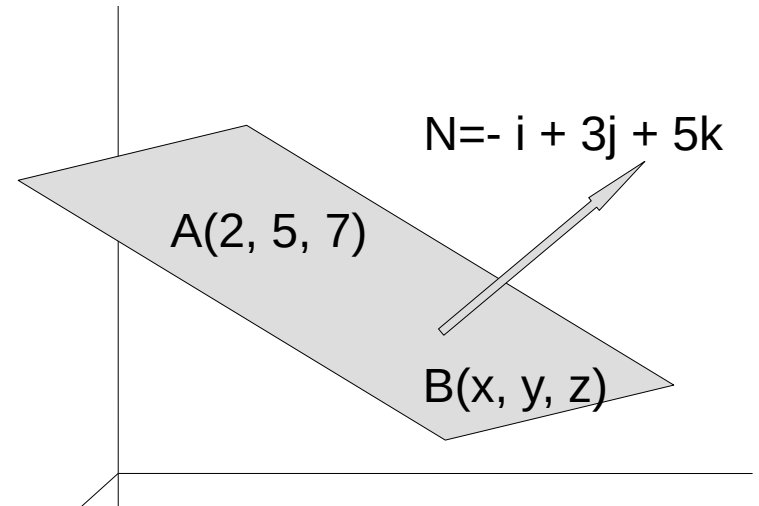
The resultant vector, **(a x b)**, is orthogonal to BOTH **a** and **b**.

We can use the right hand rule to determine the direction of **a x b**

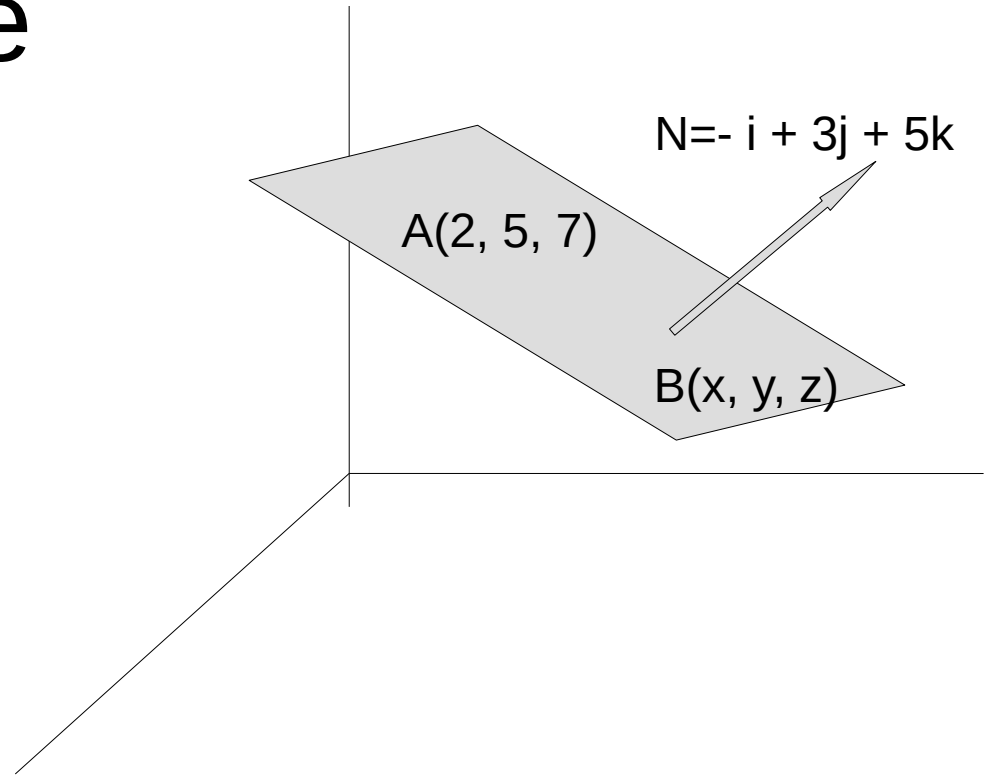


Equation of the Plane

- Here $A(2,5,7)$ is on the plane.
- N is normal to plane.
- Let $B(x,y,z)$ be on the plane.
- Vector on plane AB is $(x-2, y-5, z-7)$
- Vector $(x-2, y-5, z-7)$ and N are perpendicular to each other.
- Their dot product is
 - $-(x-2) + 3(y-5) + 5(z-7)=0$
 - $-x + 2 + 3y - 15 + 5z - 35 = 0$
 - $-x + 3y + 5z - 48 = 0$



Equation of the Plane



- This shows that the equation of any plane is in the form
- $Ax + By + Cz + D = 0$
- Where A , B , C are x , y , and z component of normal vector to the plane.

Example: Find an equation for the plane passing through the points

$$Q(-1, 1, 2), \quad R(-4, 2, 2), \quad S(-2, 1, 5).$$

Solution: when the plane passes through Q , R , and S , then the vectors

$$\overrightarrow{QR} = \langle -3, 1, 0 \rangle, \quad \overrightarrow{QS} = \langle -1, 0, 3 \rangle,$$

lie in the plane. Thus the cross-product

$$\mathbf{n} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ -3 & 1 & 0 \\ -1 & 0 & 3 \end{vmatrix} = 3\mathbf{i} + 9\mathbf{j} + \mathbf{k}$$

is normal to the plane.

If $\mathbf{r} = \langle x, y, z \rangle$ determines an arbitrary point P in the plane, the vector

$$\mathbf{v} = \overrightarrow{QP} = \langle x + 1, y - 1, z - 2 \rangle$$

lies in the plane and so is perpendicular to $\mathbf{n}$. In this case,

$$\mathbf{n} \cdot \mathbf{v} = 3(x + 1) + 9(y - 1) + (z - 2) = 0,$$

which after simplification becomes

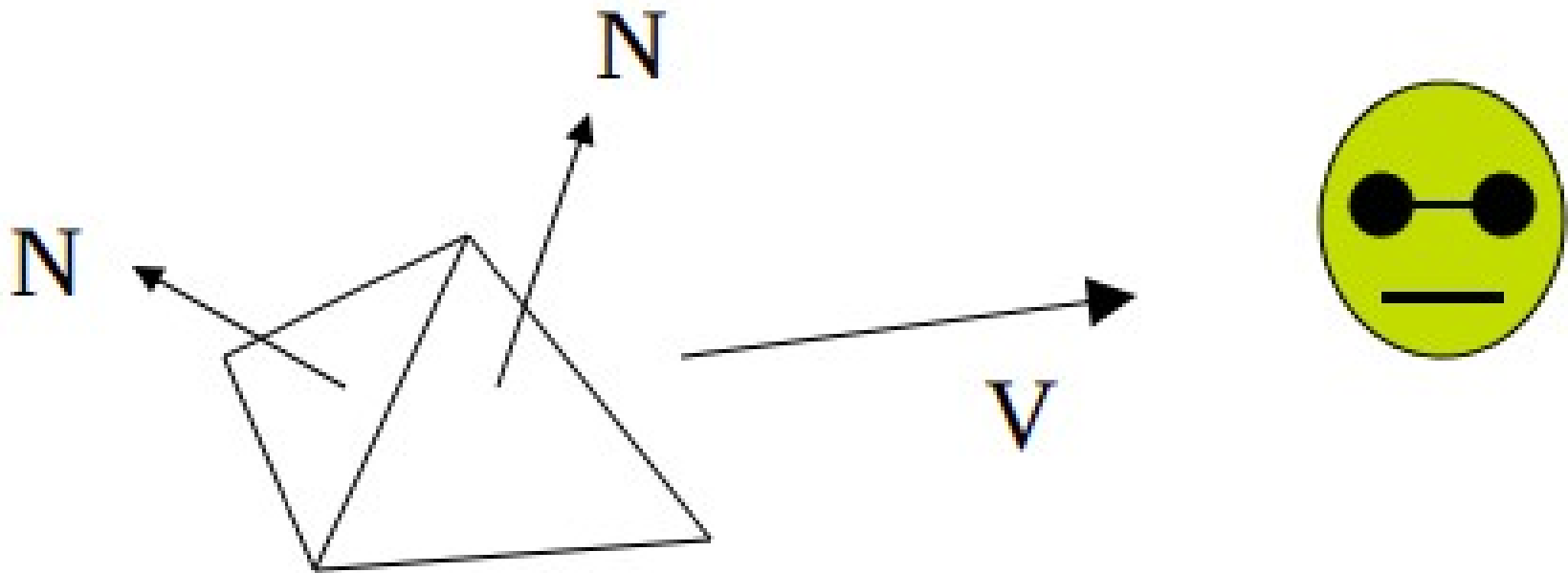
$$3x + 9y + z - 8 = 0.$$

Consequently, the plane $3x + 9y + z = 8$ passes through Q , R , and S .

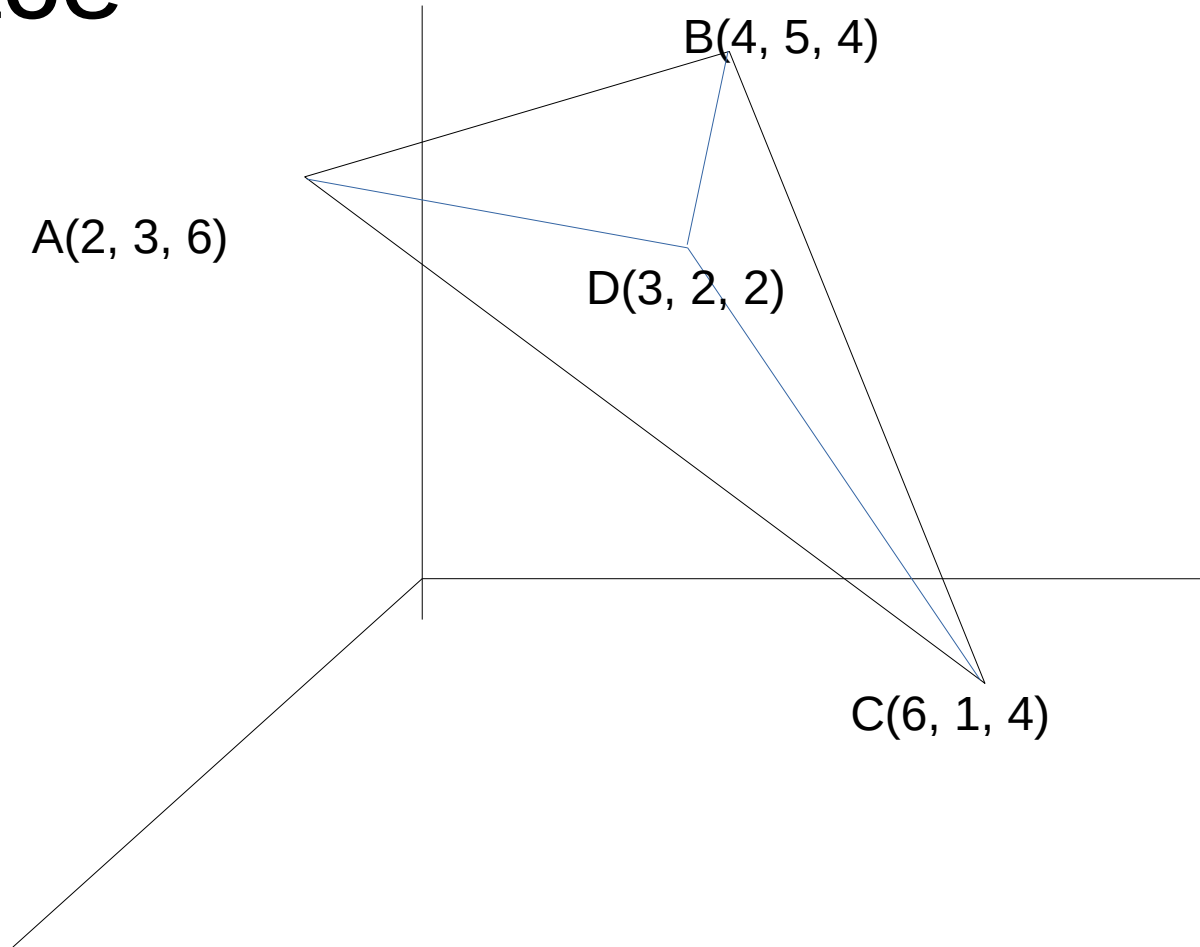
Back Face Detection

- Fast and simple
- Based on inside and outside test
- A point (x,y,z) is inside a polygon surface if
- $Ax + By + Cz + D < 0$
- Another method is to calculate $V.N$
- Where N is normal to surface and V is view vector directed towards viewer.
- $V.N > 0$, if V and N are in same direction (Front Face)
- $V.N \leq 0$, otherwise (Back Face)

Back Face



Back Face





Back Face

- We should find the normal vector from the three points on the plane.
- Two vectors on the plane can be easily found
- The cross product of these two vectors are perpendicular to each of these vectors.
- It means cross product of these two vectors are perpendicular to surface.

Back Face

- Normal Vector to ACB= $N = AB \times BC$
- $AB = OB - OA = (4, 5, 5) - (2, 3, 6) = (2, 2, -1)$
- $BC = OC - OB = (6, 1, 4) - (4, 5, 5) = (2, -4, -1)$
- $N = AB \times BC =$

$$N = \begin{vmatrix} i & j & k \\ 2 & 2 & -1 \\ 2 & -4 & -1 \end{vmatrix}$$
$$N = i \begin{vmatrix} 2 & -1 \\ -4 & -1 \end{vmatrix} - j \begin{vmatrix} 2 & -1 \\ 2 & -1 \end{vmatrix} + k \begin{vmatrix} 2 & 2 \\ 2 & -4 \end{vmatrix}$$

Back Surface

- $N=(-2-4, -2+2, -8-4)=(-6,0,-12)$
- Viewing from Z axis means $V=(0,0,1)$
- $N.V=-12 < 0$
- This is also known as z-component.
- So, ACB is not visible.
-



Back Surface

- Similarly Z component for ADB, DCB and ADC respectively are 4, 10 and -2.
- So only DCB and ADC are visible.
- This simple method can eliminate half of hidden surfaces.
- The method does not consider obscuring of surfaces by other objects.



Depth (Z) Buffer

- An image space method
- Comparisons are done at pixel level
- Uses extra buffer depth buffer (z-buffer)
- One storage for each pixel.
- Z-buffer stores distance information.
- Frame buffer stores intensity information.
-

Depth (Z) Buffer

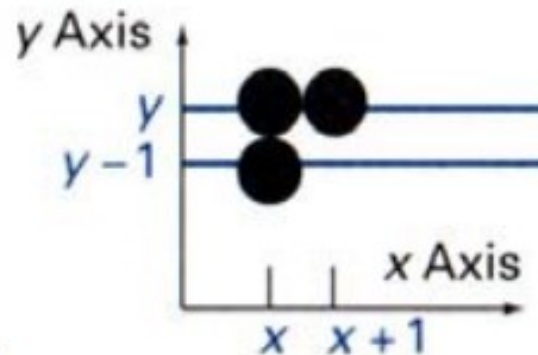
- For all (x,y)
 - $\text{depthBuffer}(x,y) = \text{infinity}$
 - $\text{frameBuffer}(x,y) = \text{bgcolor}$
- For each polygon p
 - For each position (x,y)
 - $z = \text{calculateDepth}(x,y)$
 - If $z < \text{depthBuffer}(x,y)$
 - $\text{depthBuffer}(x,y) = z$
 - $\text{frameBuffer}(x,y) = \text{Intensity}(x,y)$

Depth-Buffer: Incremental z-Values

depth at (x,y): $z = \frac{-Ax - By - D}{C}$

depth at (x+1,y): $z' = \frac{-A(x+1) - By - D}{C} = z - \frac{A}{C}$

depth at (x,y-1): $z'' = \frac{-Ax - B(y-1) - D}{C} = z + \frac{B}{C}$

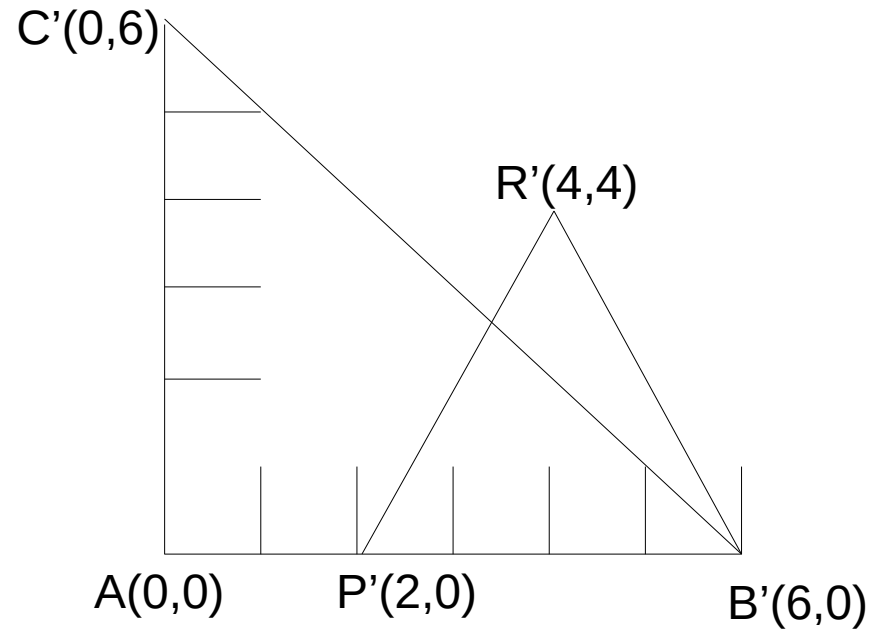


constants !



Illustrative Example

- Surface S1 is $A(0,0,6)$, $B(6,0,0)$ and $C(0,6,0)$
- S2 is $P(2,0,6)$, $Q(6,0,6)$, $R(4,4,0)$
- Projected point are
- $A'(0,0)$, $B'(6,0)$ and $C'(0,6)$
- $P'(2,0)$, $Q'(6,0)$, $R'(4,4)$
- Initialize frame Buffer and Depth Buffer as given below



∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞

Table 1: Depth Buffer

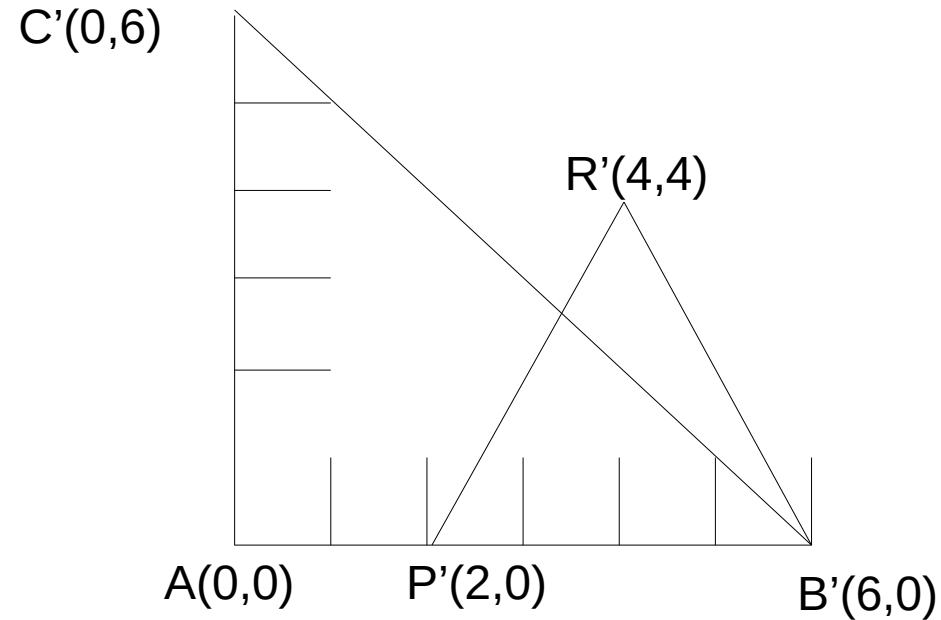
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg

Table 2: Frame Buffer

Illustrative Example

- For equation of the s_1 ,
- $AB = OB - OA = (6, 0, 0) - (0, 0, 6) = (6, 0, -6)$
- $BC = OC - OB = (0, 6, 0) - (6, 0, 0) = (-6, 6, 0)$
- Normal, $N = AB \times BC = (36, 36, 36)$
- Let us take a point $P(x, y, z)$ on S_1
- $PA = (0 - x, 0 - y, 6 - z) = (-x, -y, 6 - z)$

- We have $PA \cdot N = 0$
- Or, $(-x, -y, 6 - z) \cdot (36, 36, 36) = 0$
- Or, $-36x - 36y + 6 \cdot 36 - 36z = 0$
- Or, $x + y + z - 6 = 0$
- $A=1, B=1, C=1, D=-6$
- Surface color for $s_1 = c_1$
- Surface color for $s_2 = c_2$

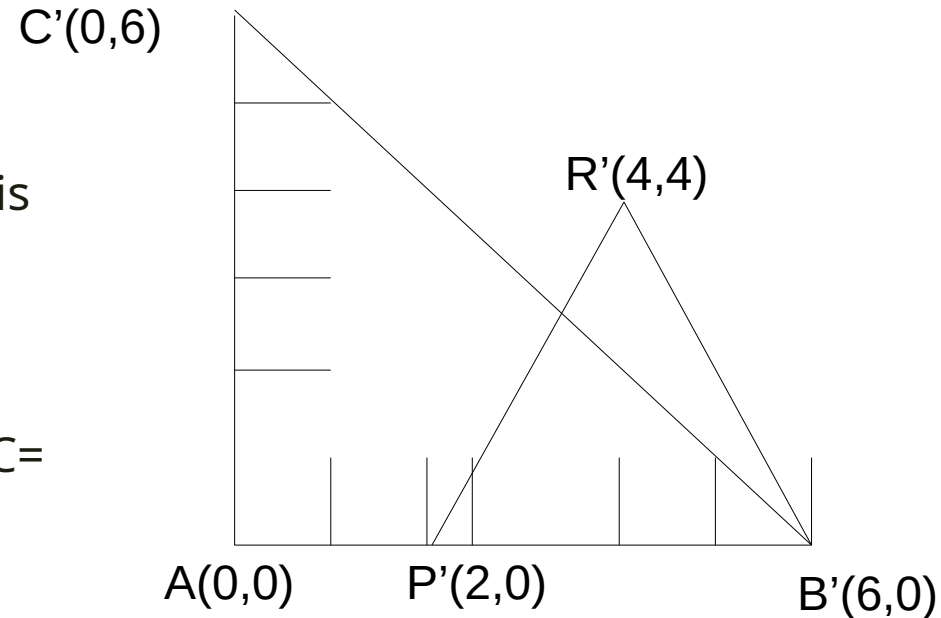


Illustrative Example

- Starting from $y=6$, the depth for $(0,6)$ is
- $z(0,6)=(-Ax-By-D)/C=(0-6+6)=0$
- Since, $z(0,6)<\text{depthBuffer}(0,6)$, $\text{depthBuffer}(0,6)=0$ and $\text{frameBuffer}(0,6)=c1$
- Now, $y=5$. The depth for $(0,5)$, $z'=z+B/C=0+1=1$
- Since $z'<\text{depthBuffer}(0,5)$, $\text{depthBuffer}(0,5)=1$ and $\text{frameBuffer}(0,5)=c1$

0	∞	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞

Table 1: Depth Buffer



c1	bg	bg	bg	bg	bg	bg
c1	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg

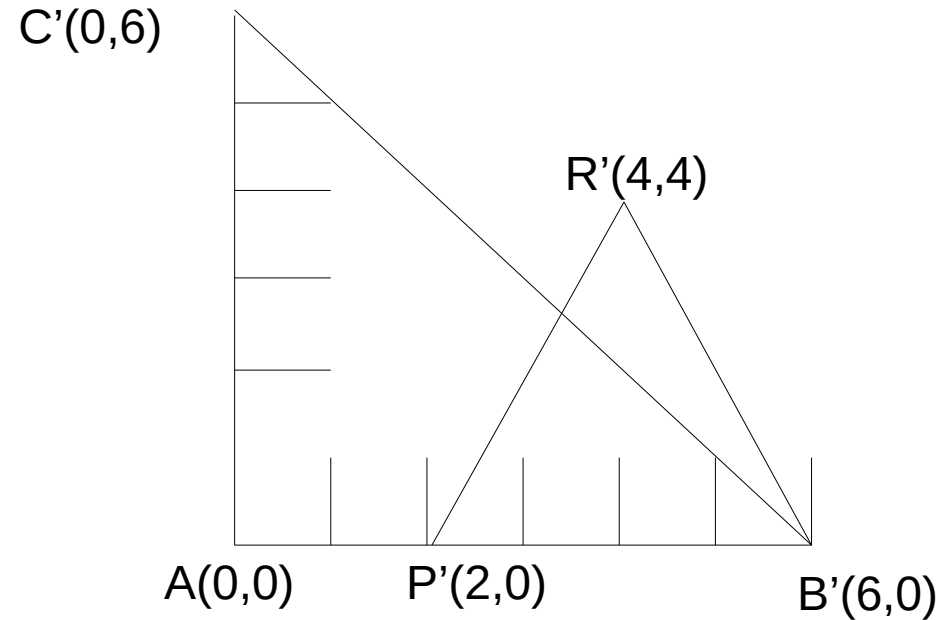
Table 2: Frame Buffer

Illustrative Example

- Depth for (1,5) is
- $z'' = z' - A/C = 1 - 1 = 0$
- Since $z'' < \text{depthBuffer}(1,5)$, $\text{depthBuffer}(1,5) = 0$ and $\text{frameBuffer}(1,5) = c1$

0	∞	∞	∞	∞	∞	∞
1	0	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞

Table 1: Depth Buffer

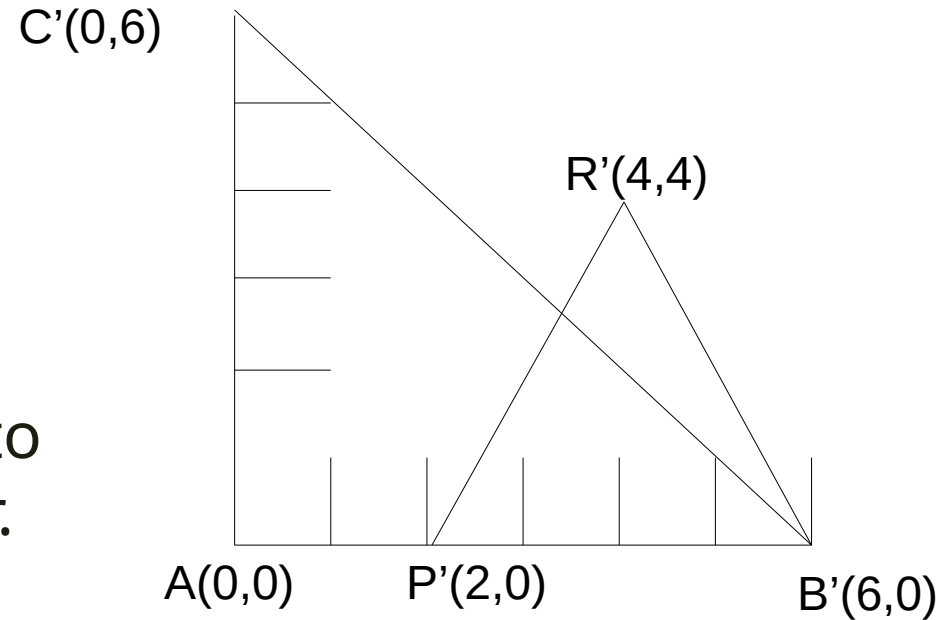


c1	bg	bg	bg	bg	bg	bg
c1	c1	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg
bg	bg	bg	bg	bg	bg	bg

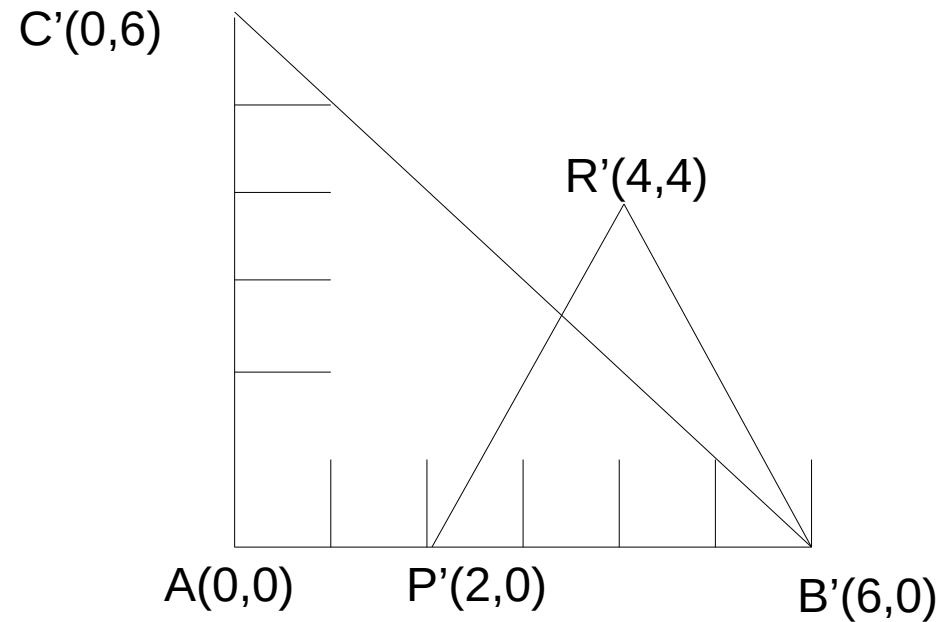
Table 2: Frame Buffer

Illustrative Example

- This is done for all the scan lines and all the surface.
- We will finally get the color to be displayed in frame buffer.
- This will remove the hidden objects.



Illustrative Example

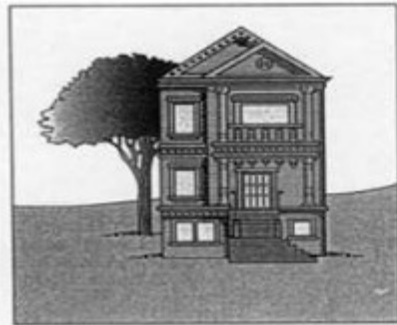


- Let us calculate the color for point $(3,1)$
- This point is shared by both $s1$ and $s2$
- In fact we should calculate color for all the point in $S1$ and $S2$
- For this we start from top most scan line
- i.e. $y=6$, So, we calculate depth of $(0,6)$ from surface $s1$.

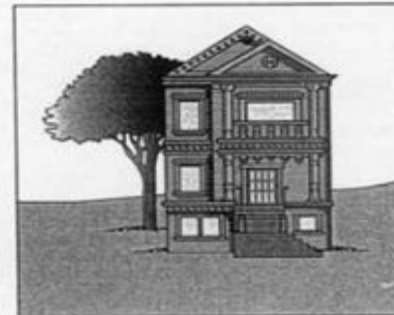
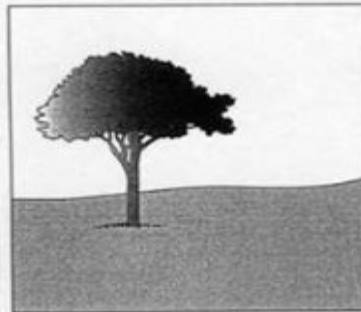
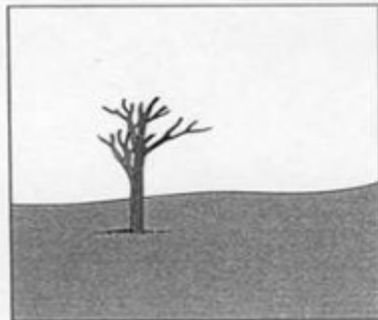
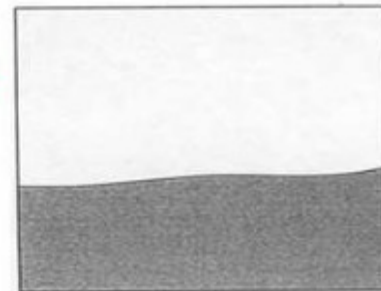
- 
- Advantages:
 - Simple and easy to implement.
 - Disadvantages:
 - Requires a lot of memory.
 - Finite depth precision can cause problems.
 - Might spend a lot of time rendering polygons that are not visible.
 - Requires re-calculations when changing the objects scale.
 - Does not do transparency easily

- **Depth Sorting Algorithms (painter's Algorithm):** Each layer of paint covers up the previous layer.

- **i.e.**, first paints the background colors
- Next, the most distant objects are added, then the nearer objects, and so forth
- At the final step, the foreground objects are painted on the canvas



By Painter's
Algorithm



Depth Sort (Painter Algorithm)

- Sort all of the polygons in the scene by their depth.
- Draw them back to front.
- **Question:** Does a depth ordering always exist? Unfortunately, no.
 - For polygons with constant Z value, this sorting clearly works.
 - For example: window systems.



Scan Line Method

- An Extension of polygon scan conversion method.
- It maintains an active list of edges that intersect the selected scan line.
- List is sorted in increasing order of x .
- For the points on selected scan line, a flag for each polygon is set to indicate that the point lies on given polygon.
- Points on the scan line is processed from left to right.
- Calculate depth if multiple polygons flag is set for the given point on scan line.
- And change the color of the point in the frame buffer with the color of the polygon with least depth.